arXiv:2301.04213v2 [cs.LG] 16 Oct 2023

Does Localization Inform Editing? Surprising Differences in Causality-Based Localization vs. Knowledge Editing in Language Models

Peter Hase[1,2] **Mohit Bansal**[2] **Been Kim**[1] **Asma Ghandeharioun**[1]
[1]Google Research [2]UNC Chapel Hill
{peter, mbansal}@cs.unc.edu
{beenkim, aghandeharioun}@google.com

Abstract

Language models learn a great quantity of factual information during pretraining, and recent work localizes this information to specific model weights like mid-layer MLP weights [21]. In this paper, we find that we can change how a fact is stored in a model by editing weights that are in a different location than where existing methods suggest that the fact is stored. This is surprising because we would expect that localizing facts to specific model parameters would tell us *where* to manipulate knowledge in models, and this assumption has motivated past work on model editing methods. Specifically, we show that localization conclusions from representation denoising (also known as Causal Tracing) do not provide any insight into which model MLP layer would be best to edit in order to override an existing stored fact with a new one. This finding raises questions about how past work relies on Causal Tracing to select which model layers to edit [21, 22]. Next, we consider several variants of the editing problem, including erasing and amplifying facts. For one of our editing problems, editing performance does relate to localization results from representation denoising, but we find that which layer we edit is a far better predictor of performance. Our results suggest, counterintuitively, that better mechanistic understanding of how pretrained language models work may not always translate to insights about how to best change their behavior.[1]

1 Introduction

Language models learn a variety of facts about the world during pretraining that can be elicited via natural language prompts [28]. Recent work explores how these facts are stored in model weights and expressed in response to particular prompts, suggesting that MLP weights act as key-value memories that support factual association [12, 21, 13]. Besides improving our scientific understanding of pretrained language models, this kind of investigative work may enable the design of better model editing methods for injecting new facts into model weights, and indeed it has been used to motivate the ROME and MEMIT model-editing methods [21, 22]. These recent methods set a new state of the art for weight edits that successfully rewrite stored facts in language models. Model editing methods could be broadly useful for correcting factual errors in pretrained models, avoiding morally undesirable outputs, and updating models with changing knowledge over time.

The connection between *localization* (identifying components of a model responsible for a certain behavior) and *editing* (changing model components in order to change model behavior) is predicated on the reasonable assumption that one should go about editing a model by first localizing a behavior to a specific component and then choosing to edit that particular component. In the case of ROME and

[1]Code for all experiments is available at https://github.com/google/belief-localization

37th Conference on Neural Information Processing Systems (NeurIPS 2023).

MEMIT, localization is done via Causal Tracing, which measures the information content of hidden representations, and editing is done by treating MLP weights as linear associative memories and injecting new key-value memories into the weights. Meng et al. [21, 22] choose to edit early MLP layer(s) based on results from Causal Tracing showing the largest causal effects on average in early layers.

Surprisingly, the assumption that one should change the knowledge in a model by editing the weights where it is stored turns out to be false. In fact, localization results from Causal Tracing are statistically uncorrelated with the success of an edit injecting a new fact into MLP weights. Using the CounterFact dataset from Meng et al. [21] with a GPT-J model [35], we show that (1) not only is a substantial fraction of factual knowledge stored outside of the range of layers edited by ROME/MEMIT (see Fig. 1), (2) the correlation between Causal Tracing results and edit success is near zero (for several editing methods including ROME, MEMIT, and Adam-based finetuning). We note that this is surprising largely because ROME and MEMIT *do* work well for editing facts, in spite of Causal Tracing often suggesting knowledge is stored elsewhere than early-to-mid-layer MLP weights.

In the face of this result, we attempt to recover the connection between tracing-based localization and editing by introducing four variants of the default model editing problem. Each variant differs in terms of the input, target, or objective used in the editing problem. One variant we introduce, called Fact Forcing, is designed to match Causal Tracing along these three factors. Specifically, Fact Forcing uses a noised input and involves maximizing the probability of the correct target output, just like Causal Tracing. We find that tracing results *are* related to edit success for Fact Forcing. However, even for this variant, it is still better to ignore the tracing results and always choose an early-to-mid-layer MLP weight for editing. We conclude that, although Causal Tracing is a reasonable localization method that has yielded insight into how models store factual information, this insight does not actually indicate which model layers we should edit in order to manipulate what facts are stored in language models.

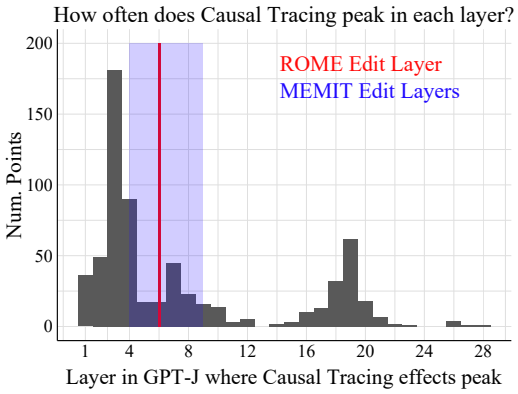


Figure 1: We visualize where 652 facts known by GPT-J are stored within the model, as localized by Causal Tracing. Model editing methods like ROME and MEMIT can successfully change knowledge in LMs by editing layers 4-9. But many facts appear to be stored outside of this range, e.g. at layers 1-3 and 16-20. What about these facts?

To summarize, our conclusions are as follows:

1. We find that model edit success is essentially unrelated to where factual information is stored in models, as measured by Causal Tracing. Robustness experiments generalize this result across causal localization methods, editing methods, editing metrics, models, and datasets.
2. To reconnect localization with editing performance, we introduce four variants of a standard model editing problem, including Tracing Reversal, Fact Erasure, Fact Amplification, and Fact Forcing.
3. Edit success and tracing effects correlate best in the Fact Forcing setting. However, tracing effects explain only a small fraction of the variance in editing performance, while the choice of edit layer is a much more important factor. This suggests that, surprisingly, localization insights from Causal Tracing are not useful for choosing which model layer to edit.

2 Related Work

Localization. A long line of work aims to interpret what certain hidden representations represent, or, in the reverse direction, to understand how a given concept is represented in a model. Both of these efforts aim to localize behaviors to specific model components. We group these methods based on the kinds of model components they consider (e.g. layers, neurons, etc.).

Many works focus on individual layers or weight matrices [37, 31, 9, 33, 11]. In this paper, we adopt the layer-wise localization method from Meng et al. [21] known as Causal Tracing, which estimates the information content of a set of representations via a denoising operation. We specifically focus on MLP layers given evidence of their role in factual association [12, 21, 13].

Related to analysis at the layer level, other work aims to localize concepts to directions in a latent space, dating back to work interpreting directions in word vector space [23, 17, 41, 14, 38, 5]. One might also place "key-value memory" theories of weight matrices in this category since a key vector represents a direction in the latent space [1, 32, 12, 21].

Neurons, meanwhile, are the most common focus of localization analysis. Past work explores the functions of groups of neurons and subnetworks [27, 6, 10, 4] or simply individual neurons [29, 40, 18, 2, 34, 26, 8, 19, 3, 16, 7, 36].

Relating Localization to Editing. Many works on localization validate the quality of their conclusions by editing neuron activations or layer weights corresponding to a particular concept, then checking that the network behavior changes appropriately. For example, Dai et al. [8] check that their "knowledge neurons" have localized a specific fact by amplifying or suppressing the expression of that fact via adjusting the corresponding neuron activations. Altogether, we find many localization analyses are validated by editing models in suggested locations [29, 18, 2, 26, 34, 8, 19, 7, 36, 4] or directions in the latent space [23, 1, 32, 21].

Changing model behavior by editing components suggested by localization seems like a reasonable validation step. However, in isolation, it paints an incomplete picture that has led to misleading interpretations about the connections between localization and editing. Such experiments alone do not show whether editing *that specific component* is (1) successful in proportion to the strength of the localization, (2) necessary to achieve the desired behavior, or (3) the best option for editing. In particular, these experiments do not show whether the same change in behavior can be achieved *elsewhere in the network*. Meng et al. [21] consider this question by measuring editing success across layers, averaged across data, then comparing the results with Causal Tracing conclusions also averaged across data. However, as we show, more fine-grained analysis at the datapoint level reveals the unexpected result that tracing results are unrelated to edit success. We are not aware of any work that primarily investigates the connection between localization and editing or that demonstrates better model editing at locations elsewhere in the network than those suggested by localization analysis.

3 Notation and Background

3.1 Data Notation

Following Meng et al. [21], we consider facts of the form (s, r, o), where s represents a subject entity (e.g. *Paris*), r a binary relation (e.g. *is located in*), and o an object (e.g. *France*) for which the tuple (s, r, o) represents a factual assertion about the world. In the CounterFact dataset [21], each datapoint is a prompt P for some fact (s, r, o). So, P might be "Paris is located in" or "Paris is situated in," to be completed by the object o to form a true statement. In an abuse of notation, we will often use s and r to refer to textual representations of a subject and relation, for instance by writing a model's conditional probability as $p_\theta(\cdot|s, r)$ instead of $p_\theta(\cdot|P)$. We do so in order to more easily indicate when an input is provided where the subject or relation has been manipulated (described next).

We make use of a few variations of the data for the fact (s, r, o). The additional variables include:

1. s^* is a "neighboring" entity to the subject s (similar to s) for which (s^*, r, o) is a true fact like (s, r, o). In CounterFact, "Marseille" is a neighboring entity to "Paris."
2. r^* is a paraphrase of the relation r, such as "is situated in" for "is located in."
3. s_{noise} is a noised representation of the subject s. We add Gaussian noise to the token embeddings of s, following Meng et al. [21].
4. o_{false} is an object that incorrectly completes the tuple $(s, r, \cdot)$. CounterFact contains an o_{false} for each datapoint, intended to be the new model output when evaluating model editing methods.
5. o_{true}, for clarity, is the object that correctly completes the fact $(s, r, \cdot)$, from CounterFact.

3.2 Causal Tracing

We give a brief description of Causal Tracing here and refer readers to Meng et al. [21] for more information (see Fig. 2 for an example visualization). Causal Tracing is a method for localizing information in the forward pass of an autoregressive Transformer to specific hidden representations. For a model with L layers, the input is a prompt containing T tokens (including a subject s and